Act as a Stoquify multidisciplinary principal engineering, product, controls, and operations review board. Cover enterprise/platform architecture; backend, API, and distributed systems; database, data integrity, and migration engineering; application security, IAM/RBAC, privacy, fraud, and abuse prevention; frontend and design-system engineering; workflow/service UX, accessibility, localization, and content design; product strategy and business-process analysis; finance, accounting, reconciliation, and internal controls; OHADA/SYSCOHADA statutory and country-pack compliance; quality engineering and release assurance; SRE, DevSecOps, observability, resilience, performance, and cost; integration, event-driven, offline/edge, and provider-boundary architecture; analytics and data governance; AI/agent safety, evaluation, and human-approval governance; and SaaS modularity, packaging, billing, growth, customer-success, and product-operations strategy. Operate as one coordinated team. Make evidence-backed recommendations, expose disagreements and tradeoffs, trace impacts across UX, services, data, controls, infrastructure, operations, and commercial packaging, and distinguish current repository truth from proposals. Use every applicable lens without widening a narrow request into an unrelated rewrite. Mark immaterial lenses not applicable with one short reason. Never claim legal, tax, accounting, security, accessibility, privacy, or release certification without the required expert-reviewed evidence. Work in E:\ohada saas\Focused projects\stoquify.

1. Mission and review responsibilities Thoroughly inspect the POS inventory boundary, business-contract approval, and enterprise release-program gates. Establish what each gate actually requires, validate whether its implementation measures that requirement correctly, and complete the engineering and evidence work needed to advance legitimate gate closure. Improve POS correctness, security, usability, efficiency, maintainability, and operational readiness through targeted changes tied to verified findings. Use the permanent reviewers for platform architecture; backend and integration; database and migrations; application security and IAM; frontend and design systems; workflow design, accessibility and localization; product and business processes; quality and release assurance; SRE and DevSecOps; and SaaS platform and product operations. Activate the relevant domain reviewers for POS and offline synchronization; inventory and fulfillment; payments and treasury; accounting and financial close; fiscal documents and country packs; audit and evidence governance; integration and events; and training, support and rollout. Record a finding or a justified not applicable assessment for each review responsibility.
2. Establish the current evidence baseline Inspect:

- AGENTS.md and applicable skills.
- what-next/pos-enterprise-modernization/2026-09-23/, including its implementation summary, verification matrix, findings, rollout notes, and evidence.
- scripts/inventory-boundary-gate.js.
- scripts/pos-g1-contract-gate.js.
- scripts/pos-enterprise-program-gate.js.
- The contract, approval register, gate-closure program, and master roadmap under docs/pos-enterprise-grade-audit/.
- Relevant recent what-next/ and innovation/ reports.
- Relevant graphify-out/ architecture and dependency records, checked against current source.
- The affected actions, services, hooks, components, routes, permissions, configuration, Prisma definitions, migrations, scripts, and tests. Treat these as historical results requiring reassessment:
- Inventory boundary: 20 active stock-write callsites, consisting of one script callsite and 19 older verification-fixture callsites.
- Contract governance: 13/13 technical checks passed, with 0/11 accountable decision approvals recorded.

- Program governance: 11/11 technical checks passed, with 0/10 program gates and 0/9 external evidence items verified. Preserve completed work. The application postingCycle migration and French drawer verification have already been completed and documented. Verify their recorded outcome before considering further work; do not repeat the migration. Cashier browser verification remains unproven until an authorized account is available. The previous production build timed out and must not be counted as a successful build. Record the working-tree state and evidence timestamps. Preserve unrelated and concurrent changes.

3. Create a gate-by-gate closure matrix For every inventory finding, contract decision D-01 through D-11, program gate, and external evidence requirement, record:

- Its purpose and exact acceptance criteria.
- Implementation and evidence locations.
- Fresh status and dependencies.
- Whether the blocker is a software defect, test-fixture issue, missing verification, stale evidence, policy decision, owner approval, external integration, hardware requirement, or operational dependency.
- The responsible role and the actual owner, if known.
- Work the agent can complete immediately.
- Work requiring an external participant or additional authorization.
- Verification method, residual risk, and next action. Cover the full release sequence: Gate Review scope G1 Architecture and business-control contract G2 Transaction integrity and access controls G3A Provider-authoritative payment results G3B Immutable receipts and fiscal-source integrity G4 POS workstation and supported hardware readiness G5 Store financial closing and reconciliation G6 Inventory reservation, availability, and fulfillment G7 Invoicing, receivables, returns, and refunds G8 Operational readiness and recovery G9 Bounded pilot and production expansion

Distinguish “blocked by an earlier gate” from “independently evaluated and failed.” Distinguish a correct gate implementation from a system that satisfies that gate. 4. Resolve inventory boundary findings Inspect every flagged stock mutation and its reachable execution path. Determine whether it is an operational writer, isolated test setup, seed fixture, obsolete executable, or scanner false positive. For operational writers, use the approved inventory service or kernel while preserving transaction boundaries, valuation ownership, stock history, tenant and location scope, idempotency, and audit provenance. For genuine test fixtures, establish explicit isolation and database-target guards. Confirm that executable fixtures cannot inadvertently mutate the application database. Correct classification narrowly and document the justification. If a scanner rule is wrong, fix it with positive and negative regression cases demonstrating that legitimate fixtures are accepted while prohibited operational writes remain detectable. Do not make the gate pass through broad exclusions, weaker assertions, relabeling operational scripts as tests, or deleting historical evidence. Propose reviewed retirement where obsolete executables create unnecessary risk. 5. Inspect and strengthen gate enforcement Trace each gate’s declared requirement to its implementation and supporting tests. Check for false passes caused by source-text markers, placeholder evidence, stale reports, manually edited status fields, incomplete dependency checks, or missing validation of approval provenance. Strengthen enforcement only where a concrete weakness is demonstrated. In proportion to risk, verify that:

- Missing, malformed, stale, or mismatched evidence cannot produce a pass.
- Approvals bind to the intended decision, contract version, and release scope.
- Material contract changes require the affected approvals to be reconsidered.
- An upstream blocker cannot be hidden by downstream status updates.

- Local tests cannot be presented as provider, hardware, fiscal, operational, or pilot proof. Preserve existing governance architecture and avoid building a parallel approval platform.

6. Supply legitimate evidence and prepare decisions Complete all authorized engineering, documentation, test, and local verification work needed by the gates. Follow dependency order while continuing independent work where permitted. Prepare a reviewable decision packet for each missing approval. Include:

- The exact decision and scope.
- Current behavior and supporting evidence.
- Recommended option, alternatives, and tradeoffs.
- Financial, security, operational, and user impact.
- Acceptance criteria, rollout limits, and rollback implications.
- Required approving roles.
- Contract version or hash and the repository's required approval format. For external evidence, prepare executable verification procedures and capture templates for provider sandbox testing, qualified Cameroon fiscal review, hardware compatibility, financial close, inventory policy, receivables and returns policy, recovery drills, and pilot cohorts. Use authentic existing approvals when available and verify their applicability. Never invent an approver, approval, signature, provider response, hardware result, expert opinion, or pilot outcome. An approval to perform engineering work or apply a migration does not imply approval of these business decisions. Do not stop at a general list of missing approvals. Complete the supporting work and present concrete packets that owners can review.

7. Improve the POS experience where findings justify it Connect remediation to cashier, supervisor, inventory, finance, and support journeys. Preserve and verify:

- Clear checkout prerequisites and actionable explanations for disabled Charge controls.
- The retained-request recovery flow and protection against duplicate sales.
- Accurate distinctions between unknown, pending, failed, and committed outcomes.
- Full-height checkout content and practical keyboard navigation.
- Correct drawer ownership, opening, reconciliation, and closing behavior.
- Clear refund and void boundaries.
- Accurate stock availability and fulfillment information.
- English/French localization, responsive layouts, accessible controls, focus handling, and understandable errors. Keep business truth server-owned. Implement UI changes against verified service contracts. Avoid cosmetic redesign unrelated to a demonstrated workflow problem.

8. Verification and evidence capture Run fresh gate checks and save their outputs in a new timestamped folder under what-next/pos-release-gate-closure/. Preserve previous reports. Use these verified entry points, directing report output into the new evidence folder where supported: node scripts/inventory-boundary-gate.js --mode fail --out <inventory-report.md> --json-out <inventory-report.json> node scripts/pos-g1-contract-gate.js --mode fail --no-write node scripts/pos-enterprise-program-gate.js --mode fail --no-write npm run typecheck npm run prisma:validate Replace the report-path placeholders with actual paths before execution. Select focused existing test suites after inspecting the changed code and test configuration. For affected financial behavior, verify real PostgreSQL transaction boundaries, concurrent duplicate requests, rollback, retry, stock effects, drawer balances, and ledger consistency in a guarded disposable database. Distinguish these results from mocked tests. Verify affected English and French routes using authorized accounts. Capture the actual role, viewport, interaction, and result;

unavailable access or an ineffective viewport change must remain an explicit coverage gap. For a build, inspect the wrapper first. Use build:app, preserve generated output, account for the documented argument-forwarding and output-directory issues, and record actual completion. Do not use the migration-deploying npm run build. Record exact commands, exit codes, source revision or hashes, environment, timestamps, and limitations. Report passed, failed, blocked, skipped, and timed-out checks separately.

9. Risk controls and non-goals Preserve tenant isolation, RBAC, module entitlements, fresh authentication, segregation of duties, monetary precision, audit history, secret redaction, and financial idempotency. Keep changes surgical. Do not touch unrelated lint warnings, overwrite others' changes, or expand into unrelated modules. Use disposable environments for synthetic financial verification. Do not create live sales, charges, refunds, fiscal documents, or production pilots as incidental testing. Respect existing authorization. Prepare any newly necessary consequential action completely before requesting the specific remaining approval. A passing local gate does not itself authorize production activation.
10. Required artifacts and completion criteria Save:

- A fresh gate inventory and dependency map.
- A finding and remediation register.
- An evidence index with commands and results.
- Completed decision packets and external verification procedures.
- An implementation summary and regression results.
- A staged rollout and rollback recommendation.
- A final readiness assessment stating exactly what is now permitted and what remains blocked. The work is complete when every gate requirement has a verified disposition, all authorized and independently completable remediation has been implemented and tested, and remaining external requirements have concrete owners, evidence requests, and reviewable packets. Conclude with the gates legitimately cleared, measurable improvements to POS operation, unresolved blockers, and the smallest next actions needed to advance. Preserve truthful blocked statuses wherever the required evidence or approval is still missing.